君正®

T23 常见问题 FAQ

Date: Jun.2023

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®
T23 常见问题 FAQ
Copyright © Ingenic Semiconductor Co. Ltd 2019. All rights reserved.

Release history

Date	Revision	Change
Jun.2023	1.0	First release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

北京君正集成电路股份有限公司

地址: 北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:(86-10)56345000
传真:(86-10)56345001
Http: //www.ingenic.com

1 概述

本文主要介绍 Zeratul T23 开发过程中常见的问题以及解决方法。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

2 目录

1 概述 1

2 目录 2

3 常见问题 3

程序无法运行、IOCTL 异常 3
3.2 Pad Normal uboot 后无法启动 4
3.3 SD 卡不能识别或热插拔 4
3.4 GPIO 冲突常见处理方法 4
3.5 编译的程序不能运行 7
3.6 无法出流问题 8
3.7 只出现一行 uboot 版本打印信息 9
3.8 修改分区后，无法启动的问题 10
3.9 音频放音延迟问题 12
3.10 编译内核出错 12
3.11 帧率不足 13
3.12 无法获得内核缓存帧 14
3.13 编译的 boot 无法运行 14
3.14 OSD 叠加 BMP 图片显示异常 17
3.15 卡起 uboot 单独更新某个分区后系统无法启动 17

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3 常见问题

程序无法运行、IOCTL 异常

如果在内核有单独的配置需求，在配置完内核之后，有可能会遇到某些句柄打开失败，或者使用出错，或者 rootfs 出现无法启动，一直重启等问题，如执行 logcat 出现以下问题：

```
# logcat
logcat read: Invalid argument
```

一般是由于改变了内核驱动程序加载的顺序，导致/dev/下设备节点的主次设备号发生改变。解决方法如下：

a）、编译 rootfs 的时候，将 build/build_camera_rootfs.sh 中创建的 mknod 删除

```
sudo mknod -m 660 ./dev/ram0 b 1 0
sudo mknod -m 660 ./dev/ttyS0 c 4 64
```

b）、修改启动脚本 etc/init.d/rcS，将 mkdev -s 提到前面，如下：

```
# !/bin/sh
# bootup_timer start >> /tmp/bootup_time
# Set Global Environment
export PATH=/system/bin:/bin:/sbin:/usr/bin:/usr/sbin
export LD_LIBRARY_PATH=/system/lib:/usr/lib
echo /sbin/mdev > /proc/sys/kernel/hotplug
/sbin/mdev -s && echo "mdev is ok....."
# Set transfer mode with MCU Environment (IIC or UART)
export TRANSFER_MODE=IIC
# Set Product mode with camera Environment (SUIT or SINGLE)
export PRODUCT_MODE=SINGLE
# Set callback script path Environment (default /usr/bin/shutdown.sh)
export CALLBACK_SCRIPT=
# Set Sensor Environment
export SENSOR=ps5270
```

c）、然后重新编译 rootfs，并将编译好的 rootfs 烧录到设备上，正常起来一下，ls -hl /dev 看下具体的主次设备号，并将必要的节点重新填写到 rootfs 编译脚本中（build_camera_rootfs.sh），再次编译 rootfs 并烧录即可，如下：

```
sudo mknod -m 660 ./dev/console c 5 1
sudo mknod -m 660 ./dev/dsp c 14 3
sudo mknod -m 660 ./dev/framechan0 c 10 63
sudo mknod -m 660 ./dev/framechan1 c 10 62
sudo mknod -m 660 ./dev/framechan2 c 10 61
sudo mknod -m 660 ./dev/i2c-2 c 89 2
sudo mknod -m 660 ./dev/ipu c 10 58
sudo mknod -m 660 ./dev/isp-m0 c 10 59
sudo mknod -m 660 ./dev/jz_adc_aux_0 c 10 55
sudo mknod -m 660 ./dev/log_main c 10 57
sudo mknod -m 660 ./dev/mem c 1 1
sudo mknod -m 660 ./dev/null c 1 3
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.2 Pad Normal uboot 后无法启动

T23 uboot 默认分区大小为 256K，其中前面 32K 为 bootload 大小，Normal uboot 需要 pad 到 32K 之后的位置，如果出现起不来的情况，需要注意：

1、Normal uboot 的大小是否有超出分区范围。

2、Pad 位置是否为 32K 位置。

3.3 SD 卡不能识别或热插拔

新做的开发版，经常出现 SD 卡无法识别，或者出现刚开机可以识别，但是无法热插拔的情况。

一般情况需要检查 SD 卡的供电是否正常，SDIO 的 GPIO 功能是否正常。以及对应的 CD-N 探测 PIN 设置是否正常。

代码如下位置 arch/mips/xburst/soc-t23/chip-t23/isvp/Indus_T23_QFN/board.h

```
/* ****************************GPIO MMC START********************************* */
#define GPIO_MMC_RST_N          -1
#define GPIO_MMC_RST_N_LEVEL    LOW_ENABLE
#define GPIO_MMC_CD_N           GPIO_PB(29)
#define GPIO_MMC_CD_N_LEVEL     LOW_ENABLE
#define GPIO_MMC_PWR            -1
#define GPIO_MMC_PWR_LEVEL      HIGH_ENABLE
#define GPIO_MMC_WP_N           -1
#define GPIO_MMC_WP_N_LEVEL     LOW_ENABLE
/* ****************************GPIO MMC END********************************* */
```

3.4 GPIO 冲突常见处理方法

一般在内核启动过程中，遇到以下堆栈信息的时候，不用慌张，只是一个警告，此时系统一般是可以正常运行的，这个打印其实是 GPIO 冲突了，打印出了警告信息。

```
)00187] Mount-cache hash table entries: 512
)00638] Initializing cgroup subsys debug
)00660] Initializing cgroup subsys freezer
)02352] NET: Registered protocol family 16
)02729] register process exit failed.
)04880] bio: create slab <bio-0> at 0
)06266] jz-dma jz-dma: JZ SoC DMA initialized
)06455] gpio:jz->reg = 0xb0011000
)06462] gpio pin: 0x400000
)06466] jz->dev_map[0]: 0x7fe81f
)06476] CPU: 0 PID: 1 Comm: swapper Not tainted 3.10.14-Archon #1
)06482] Stack : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
)06482]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
)06482]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
)06482]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
)06482]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 8202bc40
)06482]    ...
)06552] Call Trace:[<8002246c>] 0x8002246c
)06566] [<8002246c>] 0x8002246c
)06572] [<80019a50>] 0x80019a50
)06577] [<804a4218>] 0x804a4218
)06582] [<801b7718>] 0x801b7718
)06586] [<80154040>] 0x80154040
)06591] [<8019e1d4>] 0x8019e1d4
)06596] [<80215efc>] 0x80215efc
)06601] [<804a4218>] 0x804a4218
)06606] [<801dcf5c>] 0x801dcf5c
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.4.1 如何确定是哪个 GPIO 冲突

主要通过以下两行打印来判断。

gpio:jz->reg = 0xb0011000

gpio pin: 0x400000

gpio:jz->reg 代表寄存器基址，对应关系如下：

0xb0010000	GPIO_PA
0xb0011000	GPIO_PB
0xb0012000	GPIO_PC
0xb0013000	GPIO_PD

gpio pin: 代表对应的 pin 脚的 map，如 0x400000， bit22 为 1，所以通过上面分析，应该就是 GPIO_PB(22) 冲突了。

3.4.2 如何解决冲突

首先，打开 arch/mips/xburst/soc-t23/include/mach/platform.h 文件，在文件中找对应 PB22 相关联的宏定义，都需要找出来。

如：UART0_PORTB

```
/*****************************************************************************************************/
#define UART0_PORTB                                                                                 \
    { .name = "uart0", .port = GPIO_PORT_B, .func = GPIO_FUNC_0, .pins = 0x9 << 19, }
#define UART1_PORTB                                                                                 \
    { .name = "uart1", .port = GPIO_PORT_B, .func = GPIO_FUNC_0, .pins = 0x3 << 23, }
#define UART2_PORTC                                                                                 \
    { .name = "uart2", .port = GPIO_PORT_C, .func = GPIO_FUNC_2, .pins = 0x3 << 13, }
/*****************************************************************************************************/

#define MSC0_PORTB_4BIT                                                                             \
    { .name = "msc0-pb-4bit",    .port = GPIO_PORT_B, .func = GPIO_FUNC_0, .pins = (0x3f<<0), }
#define MSC1_PORTA_6BIT                                                                             \
    { .name = "msc1-pb-6bit",    .port = GPIO_PORT_A, .func = GPIO_FUNC_3, .pins = (0xf00 | 0x30000), }

#define MSC1_PORTB                                                                                  \
    { .name = "msc1-pB",         .port = GPIO_PORT_B, .func = GPIO_FUNC_1, .pins = (0x6f<<8), }
#define MSC1_PORTC                                                                                  \
    { .name = "msc1-pC",         .port = GPIO_PORT_C, .func = GPIO_FUNC_0, .pins = (0x3f<<2), }
#define I2S_PORTC                                                                                   \
    {.name = "i2s",               .port = GPIO_PORT_C, .func = GPIO_FUNC_0, .pins = (0x7f << 18),}

/*****************************************************************************************************/
/*****************************************************************************************************/

#define SSI0_PORTC                                                                                  \
    { .name = "ssi0-pc",           .port = GPIO_PORT_C, .func = GPIO_FUNC_0, .pins = (0x3 << 8 | 0xf << 11), }
#define SSI1_PORTB_1                                                                                \
    { .name = "ssi1-pb-1",         .port = GPIO_PORT_B, .func = GPIO_FUNC_1, .pins = (0x3 << 17 | 0x3 << 25), }
#define SSI1_PORTB_2                                                                                \
    { .name = "ssi1-pb-2",         .port = GPIO_PORT_B, .func = GPIO_FUNC_2, .pins = (0xf << 27), }
#define SSISLV_PORTB                                                                                \
    { .name = "ssi-slave-pb",      .port = GPIO_PORT_B, .func = GPIO_FUNC_1, .pins = (0x3 | 0x3 << 4), }
#define SSISLV_PORTC                                                                                \
    { .name = "ssi-slave-pc",      .port = GPIO_PORT_C, .func = GPIO_FUNC_1, .pins = (0xf << 2), }
/* SFC : the function of hold and reset not use, hardware pull up */
#define SFC_PORTA_QUAD                                                                              \
    { .name = "sfc",        .port = GPIO_PORT_A, .func = GPIO_FUNC_1, .pins = (0x3f << 23), }
#define SFC_PORTA                                                                                   \
    { .name = "sfc",        .port = GPIO_PORT_A, .func = GPIO_FUNC_1, .pins = (0x3 << 23) | (0x3 << 27), }
```

然后再打开 arch/mips/xburst/soc-t23/common/platform.c 文件，查看 UART0_PORTB 是由哪个 config 来决定的，发现是 CONFIG_SERIAL_JZ47XX_UART0 来决定的。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
#ifdef CONFIG_I2C1_PB25_PB26
    I2C1_PORTB,
#endif
#ifdef CONFIG_I2C1_PC08_PC09
    I2C1_PORTC,
#endif

#ifdef CONFIG_SOC_MCLK
    MCLK_PORTA,
#endif
#ifdef CONFIG_SERIAL_JZ47XX_UART0
    UART0_PORTB,
#endif
#ifdef CONFIG_SERIAL_JZ47XX_UART1
    UART1_PORTB,
#endif
#ifdef CONFIG_SERIAL_JZ47XX_UART2
    UART2_PORTC,
#endif
#ifdef CONFIG_JZ_PWM_GPIO_B17
    PWM_PORTB_BIT17,
#endif
#ifdef CONFIG_JZ_PWM_GPIO_B18
    PWM_PORTB_BIT18,
#endif
#ifdef CONFIG_JZ_PWM_GPIO_C8
    PWM_PORTC_BIT8,
#endif
```

再打开内核目录下的 .config，确认一下这个宏是否有定义，如果有定义，通过 make menuconfig 来去掉即可。

```
#
CONFIG_SERIAL_CORE=y
CONFIG_SERIAL_CORE_CONSOLE=y
# CONFIG_SERIAL_SCCNXP is not set
# CONFIG_SERIAL_TIMBERDALE is not set
# CONFIG_SERIAL_ALTERA_JTAGUART is not set
# CONFIG_SERIAL_ALTERA_UART is not set
# CONFIG_SERIAL_ARC is not set
CONFIG_SERIAL_JZ47XX_UART=y
CONFIG_SERIAL_JZ47XX_CONSOLE=y
CONFIG_SERIAL_JZ47XX_UART0=y
CONFIG_SERIAL_JZ47XX_UART1=y
# CONFIG_SERIAL_JZ47XX_UART1_DMA is not set
# CONFIG_SERIAL_JZ47XX_UART2 is not set
# CONFIG_SERIAL_JZ47XX_UART3 is not set
# CONFIG_SERIAL_JZ47XX_UART4 is not set
# CONFIG_TTY_PRINTK is not set
# CONFIG_IPMI_HANDLER is not set
# CONFIG_HW_RANDOM is not set
# CONFIG_R3964 is not set
# CONFIG_RAW_DRIVER is not set
# CONFIG_TCG_TPM is not set
```

但是有些情况，通过这个方式发下.config 中并没有定义，那该怎么办呢，有可能是驱动程序某些位置直接申请了 GPIO，需要打开 arch/mips/xburst/soc-t23/common/gpio.c 文件，再 jz_gpio_request 函数中增加如下调试信息，主要目的就是将申请者的堆栈信息打印出来，如果重复申请，这里必定会打印两次堆栈信息，然后我们通过堆栈信息来找到函数调用关系，并找到是哪个位置申请了 GPIO 即可。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
static int jz_gpio_request(struct gpio_chip *chip, unsigned offset)
{
    struct jzgpio_chip *jz = gpio2jz(chip);

    if(((unsigned int)jz->reg == 0xb0011000) && (1 << offset) == 0x400000) {
        printk("Check GPIO Request !!!\n");
        dump_stack();
    }

    if(!test_bit(offset, jz->gpio_map)) {
        printk(KERN_WARNING "gpio has conflict\n");
        return -EINVAL;
    }
    if(jz->dev_map[0] & (1 << offset)) {
        printk("gpio:jz->reg = 0x%x\n", (unsigned int)jz->reg);
        printk("gpio pin: 0x%x\n", 1 << offset);
        printk("jz->dev_map[0]: 0x%x\n", (unsigned int)jz->dev_map[0]);
        dump_stack();
        printk("%s:gpio functions has redefinition", __FILE__);
    }
    jz->dev_map[0] |= 1 << offset;

    /* Disable pull up/down as default */
    gpio_set_func(jz,GPIO_PULL_HIZ,BIT(offset));

    clear_bit(offset, jz->gpio_map);
    return 0;
}
```

3.5 编译的程序不能运行

编译的程序不能运行，出现如下打印信息，如：

```
[root@Zeratul:tmp]# ./test
-sh: ./test: not found
[root@Zeratul:tmp]#
```

```
./test: line 1: syntax error
unexpected word (expecting ")")
```

一般情况是由于编译的程序与系统没有对上导致的。可以执行 readelf -d test 来查看 test 为可执行程序名字。

```
wqyan@wqyan-VirtualBox:/media/sf_share$ readelf -d test

Dynamic section at offset 0x17c contains 30 entries:
  标记        类型                         名称/值
 0x00000001 (NEEDED)                     共享库: [librt.so.1]
 0x00000001 (NEEDED)                     共享库: [libc.so.6]
 0x0000000c (INIT)                       0x40062c
 0x0000000d (FINI)                       0x400ae0
 0x00000004 (HASH)                       0x400294
 0x00000005 (STRTAB)                     0x40045c
 0x00000006 (SYMTAB)                     0x40032c
 0x0000000a (STRSZ)                      273 (bytes)
 0x0000000b (SYMENT)                     16 (bytes)
 0x70000016 (MIPS_RLD_MAP)               0x410c80
 0x00000015 (DEBUG)                      0x0
 0x00000003 (PLTGOT)                     0x410c90
 0x00000011 (REL)                        0x4005d4
 0x00000012 (RELSZ)                      16 (bytes)
 0x00000013 (RELENT)                     8 (bytes)
```

glibc

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
wqyan@wqyan-VirtualBox:/media/sf_share$ readelf -d test

Dynamic section at offset 0x160 contains 27 entries:
  标记        类型                         名称/值
 0x00000001 (NEEDED)                     共享库: [librt.so.0]
 0x00000001 (NEEDED)                     共享库: [libc.so.0]
 0x0000000c (INIT)                       0x40056c
 0x0000000d (FINI)                       0x400a00
 0x00000004 (HASH)                       0x400260
 0x00000005 (STRTAB)                     0x400450
 0x00000006 (SYMTAB)                     0x400300
 0x0000000a (STRSZ)                      193 (bytes)
 0x0000000b (SYMENT)                     16 (bytes)
 0x70000016 (MIPS_RLD_MAP)               0x410ba0
 0x00000015 (DEBUG)                      0x0
 0x00000003 (PLTGOT)                     0x410bb0
 0x00000011 (REL)                        0x400514
 0x00000012 (RELSZ)                      16 (bytes)
 0x00000013 (RELENT)                     8 (bytes)
 0x70000001 (MIPS_RLD_VERSION)           1
```

uclibc

3.5.1 如何进行 glibc 和 uclibc 编译

toolchain 包含了 glibc 和 uclibc，因此基于 glibc 或者 uclibc 的程序均可使用此 toolchan 进行编译。

- glibc 程序编译方法：默认 link 的 libc 即为 glibc
- uclibc 程序编译方法：C_FLAGS+=-muclibc CXX_FLAGS+=-muclibc，LD_FLAGS+=-muclibc
- uclibc 程序编译也可以直接使用 mips-linux-uclibc-gnu-开头的编译工具来编译，这样 FLAGS 可以不用再添加-muclibc

3.6 无法出流问题

如果遇到系统无法出流，且出现以下打印，这个主要原因是由于 cmdline 中的 nrvbs 参数与应用程序代码的 nrvbs 参数没有匹配导致。

```
[  103.578108] error handler!!!
[  103.598383] qbuf: buffer index(3) out of range(0)
[  103.603571] CPU: 0 PID: 104 Comm: sample-Encoder- Not tainted 3.10.14-Archon #5
[  103.611160] Stack : 00000000 8003f4f8 00000000 80460000 00000000 00000000 00000000 00000000
[  103.611160]    00000000 00000000 804f2d8a 00000043 820acf58 00483ab4 00000000 00000000
[  103.611160]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
[  103.611160]    00000000 00000000 00000000 00000000 00000000 80613c00 80613cdc 8041077c
[  103.611160]    8045c1a7 80040d24 00002000 8041077c 00000000 00000068 820acf58 80613c50
[  103.611160]    ...
[  103.647920] Call Trace:
[  103.650451] [<8002239c>] show_stack+0x64/0x7c
[  103.654958] [<803ad7a8>] dump_stack+0x20/0x2c
[  103.659452] [<80344b38>] isp_printf+0x88/0x90
[  103.663952] [<8034734c>] frame_channel_vb2_qbuf+0x25c/0x484
[  103.669701] [<80347ca0>] frame_channel_unlocked_ioctl+0x318/0x668
[  103.676004] [<800f6cbc>] do_vfs_ioctl+0xa8/0x7f8
[  103.680770] [<800f745c>] SyS_ioctl+0x50/0x94
[  103.685186] [<8002531c>] stack_done+0x20/0x44
[  103.689677]
[  103.691987] Streaming off, will not wait for buffers,chan index 0
[  103.701491] CPU: 0 PID: 112 Comm: Framesource-0 Not tainted 3.10.14-Archon #5
[  103.709845] Stack : 00000000 8003f4f8 00000000 80460000 00000000 00000000 00000000 00000000
[  103.709845]    00000000 00000000 804f2d8a 00000041 820ad618 80540000 00000000 00000000
[  103.709845]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
[  103.709845]    00000000 00000000 00000000 00000000 00000000 806c7d00 806c7d54 8041077c
[  103.709845]    8045c1a7 80040d24 fffffff3 8041077c 00000000 00000070 820ad618 806c7cc8
[  103.709845]    ...
[  103.746567] Call Trace:
[  103.749089] [<8002239c>] show_stack+0x64/0x7c
[  103.753595] [<803ad7a8>] dump_stack+0x20/0x2c
[  103.758095] [<80344b38>] isp_printf+0x88/0x90
[  103.762599] [<80347f7c>] frame_channel_unlocked_ioctl+0x5f4/0x668
[  103.768890] [<800f6cbc>] do_vfs_ioctl+0xa8/0x7f8
[  103.773662] [<800f745c>] SyS_ioctl+0x50/0x94
[  103.778065] [<8002531c>] stack_done+0x20/0x44
[  103.782566]
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

如果遇到系统无法出流，且出现以下打印，这个主要原因是由于 cmdline 中的 Sensor 的宽高参数与应用程序代码的 Sensor 宽高参数没有匹配导致。

```
[    8.749087] [<8002239c>] show_stack+0x64/0x7c
[    8.753582] [<803ad7a8>] dump_stack+0x20/0x2c
[    8.758068] [<80344b38>] isp_printf+0x88/0x90
[    8.762575] [<80347ca0>] frame_channel_unlocked_ioctl+0x318/0x668
[    8.770882] [<800f6cbc>] do_vfs_ioctl+0xa8/0x7f8
[    8.775706] [<80347330>] frame_channel_vb2_qbuf+0x240/0x484
[    8.781468] [<800f745c>] SyS_ioctl+0x50/0x94
[    8.785876] [<8002531c>] stack_done+0x20/0x44
[    8.790361]
[    8.791902] [<80347ca0>] frame_channel_unlocked_ioctl+0x318/0x668
[    8.798187] [<800f6cbc>] do_vfs_ioctl+0xa8/0x7f8
[    8.803050] [<800f745c>] SyS_ioctl+0x50/0x94
[    8.807463] [<8002531c>] stack_done+0x20/0x44
[    8.811991]
[    8.836688] qbuf: invalid memory size, length = 3538944 sizeimage = 2350080
[    8.843927] CPU: 0 PID: 79 Comm: sample-Encoder- Not tainted 3.10.14-Archon #5
[    8.851398] Stack : 00000000 8003f4f8 00000000 80460000 00000000 00000000 00000000 00000000
[    8.851398]    00000000 00000000 804f2d8a 00000042 806cc1d8 00000000 00000000 00000000
[    8.851398]    00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
[    8.851398]    00000000 00000000 00000000 00000000 00000000 807f5c00 807f5cdc 8041077c
[    8.851398]    8045c1a7 80040d24 804b0da0 8041077c 00000000 0000004f 806cc1d8 807f5c50
[    8.851398]    ...
[    8.888067] Call Trace:
[    8.890596] [<8002239c>] show_stack+0x64/0x7c
[    8.895112] [<803ad7a8>] dump_stack+0x20/0x2c
[    8.900678] [<80344b38>] isp_printf+0x88/0x90
[    8.905230] [<80347330>] frame_channel_vb2_qbuf+0x240/0x484
[    8.910984] [<80347ca0>] frame_channel_unlocked_ioctl+0x318/0x668
[    8.917283] [<800f6cbc>] do_vfs_ioctl+0xa8/0x7f8
[    8.922058] [<800f745c>] SyS_ioctl+0x50/0x94
[    8.926468] [<8002531c>] stack_done+0x20/0x44
[    8.930956]
```

3.7 只出现一行 uboot 版本打印信息

在调试过程中，如果只出现了一行 uboot 版本信息打印，有时会出现自动重启，循环的打印 uboot 版本信息，如下图：

```
Ver:20200401-Turret
```

这种情况一般是卡死在内核，或者内核出现了异常，导致无法启动，需要编译 debug 版本进行内核调试，查找具体原因。

1、内核需要把 early printk 打开，如下，通过 make menuconfig 选择。然后重新编译内核即可。

注意：单独编译内核，编译脚本不需要加 -f 参数，否则回重新覆盖原有配置，编译后最好再次检查一下.config 中配置信息是否被覆盖。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
< > Red-Black tree test
< > Interval tree test
[ ] Enable dynamic printk() support
[ ] Enable debugging of DMA-API usage
[ ] Perform an atomic64_t self-test at boot
[ ] Sample kernel code  --->
[ ] KGDB: kernel debugger  --->
< > Test functions located in the string_helpers module at runtime
< > Test kstrto*() family of functions at runtime
[*] Early printk
[ ] Built-in kernel command line
[ ] Check for stack overflows
[ ] Enable run-time debugging
[ ] Enable compressed kernel support debugging
[ ] Enable spinlock timing tests in debugfs
```

2、重新 source 环境变量，选择 debug 模式，然后重新编译 tag 分区。

重新制作完 TAG、Kernel 之后，烧录到板子上，再次启动查看对应打印信息，然后分析即可。

3.8 修改分区后，无法启动的问题

正常快起的分区信息一般由 cmdline 来决定，recovery 系统的分区一般由 pad 的 uboot 来决定。

默认 uboot 分区大小为 256K，如果 uboot 分区大小需要调整，则需要原厂重新提供调整过分区的 SPL 固件。建议 uboot 分区尽量不要做调整。

3.8.1 正常系统下分区调整

1、正常系统的 cmdline 由 tag 决定，所以需要修改 tag 制作脚本，tools/make_tag/tag_generator/build_tag_t23.sh

2、修改./build/build_camera_all.sh，这个脚本会最终将各个分区打包到一个固件中，所以该分区信息也需要对应修改。

3.8.2 Recovery 系统下的分区调整

Recovery 系统下的分区由 pad 的 uboot 代码决定，os/uboot/u-boot/include/configs/isvp_t23.h 文件中。修改如下信息即可：

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
336
337 #ifdef CONFIG_SPL_MMC_SUPPORT
338     #define CONFIG_BOOTARGS BOOTARGS_COMMON " init=/linuxrc root=/dev/mmcblk0p2 rw rootdelay=1"
339 #elif defined(CONFIG_SFC_NOR)
340     #define CONFIG_BOOTARGS BOOTARGS_COMMON " init=/linuxrc rootfstype=squashfs root=/dev/mtdblock2 rw mtdparts=jz_sfc:256k(boot),2560k(kernel),2048k(root),-(appfs)"
341 #elif defined(CONFIG_SFC_NAND)
342     #define CONFIG_BOOTARGS BOOTARGS_COMMON " ip=off init=/linuxrc ubi.mtd=2 root=ubi0:rootfs ubi.mtd=3 rootfstype=ubifs rw"
343 #endif
344
345 /**
346  * Boot command definitions.
347  */
348 #define CONFIG_BOOTDELAY 1
349
350 #ifdef CONFIG_SPL_MMC_SUPPORT
351 #define CONFIG_BOOTCOMMAND "mmc read 0x80600000 0x1800 0x3000; bootm 0x80600000"
352 #endif
353
354 #ifdef CONFIG_SFC_NOR
355     #define CONFIG_BOOTCOMMAND "sf probe;sf read 0x80600000 0x40000 0x280000; bootm 0x80600000"
356 #endif /* CONFIG_SFC_NOR */
357
358 #ifdef CONFIG_SFC_NAND
359     #define CONFIG_BOOTCOMMAND "sfcnand read 0x100000 0x400000 0x80600000 ;bootm 0x80600000"
360 #endif
361
362 /**
363  * Drivers configuration.
364  */
365 /* MMC */
366 #if defined(CONFIG_JZ_MMC_MSC0) || defined(CONFIG_JZ_MMC_MSC1)
367 #define CONFIG_GENERIC_MMC     1
368 #define CONFIG_MMC             1
~/work/zeratul/os/uboot/u-boot/include/configs/isvp_t31.h [POS=344,1][50%]31/08/20 - 15:15
```

分区信息

启动命令

3.8.3 修改分区后，无法启动

修改分区一般通过 tag 中的 cmdline 信息来修改，但是经常会遇到修改完分区后出现无法启动的问题。

主要可能会有两方面的问题：

1、tag 中 cmdline 中分区信息修改了，但是打包固件的脚本未修改，打包固件还是按原有方式打包，导致无法启动，此时需要将打包固件分区对应也进行修改。

需要将 ./build/build_camera_all.sh 中的分区偏移也进行修改。

```
10 # Setup the variables
11
12 BIN_NAME=ZRT_CAMERA_FW.bin
13
14 BOOT_OFFSET=0
15 BOOT_SIZE_KBYTES=256
16 TAG_OFFSET=256
17 TAG_SIZE_KBYTES=352
18 KERNEL_OFFSET=608
19 KERNEL_SIZE_KBYTES=5120
20 ROOTFS_OFFSET=5728
21 ROOTFS_KBYTES=3072
22 RECOVERY_OFFSET=8800
23 RECOVERY_KBYTES=2560
24 SYSTEM_OFFSET=11360
25 SYSTEM_SIZE_KBYTES=4512
26 FLASH_TOTAL_SIZE=16384
```

2、在 cmdline 中还存在一个 bootinfo 参数，也需要对应修改，很多情况有可能这里会忘记修改，导致系统无法启动。

```
if [ ${ZRT_ENV_SOC_TYPE} = SOC_T31Z ]; then
    CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdinit=/linuxrc
mtdparts=jz_sfc:256K(boot),352K(tag),5M(kernel),3M(rootfs),2560K(recovery),4512K(system),512k(config),16M@0(all)
lpj=6955008 ${QUIET}"
    BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
    ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;eenv;"
    SOC_TYPE=t31
    RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
fi
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.9 音频放音延迟问题

在使用过程中，会存在音频 AO 放音的时候，出现塞了很多数据到 AO 通道，但是并没有通过喇叭播放出来。

在开启 AO 的时候，会配置一些参数，如下，其中 frmNum 指的是 SDK 中缓存 buffer 数据，numPerFrm 则是每一个 buffer 存放的采样点的个数。

出现播放延迟主要是由于 AO 有 Cache 功能，该功能主要为了防止网络抖动，导致放音断续的问题。所以在播放的时候，实际的音频数据需要塞满 frmNum 的一半才会真正播放。所以就表现出放音延迟的问题。

```
/* Step 1: set public attribute of AO device. */
int devID = 0;
IMPAudioIOAttr attr;
attr.samplerate = AUDIO_SAMPLE_RATE_16000;
attr.bitwidth = AUDIO_BIT_WIDTH_16;
attr.soundmode = AUDIO_SOUND_MODE_MONO;
attr.frmNum = 20;
attr.numPerFrm = 640;
attr.chnCnt = 1;
ret = IMP_AO_SetPubAttr(devID, &attr);
if (ret != 0) {
    IMP_LOG_ERR(TAG, "set ao %d attr err: %d\n", devID, ret);
    return NULL;
}
```

在 SDK 中，提供了一个 IMP_AO_CacheSwitch API，该接口主要用于开关 Cache 功能。如果需要播放及时，那么需要把 Cache 功能关闭，调用以下 API 即可。

```
/**
 * @fn int IMP_AO_CacheSwitch(int audioDevId, int aoChn, int cache_en)
 * 关闭音频播放缓存机制
 * @param[in] audioDevId 音频设备号.
 * @param[in] aoChn 音频输出通道号.
 * @param[in] cache_en 缓存机制开启开关
 *
 * @retval 0 成功.
 * @retval 非0 失败.
 *
 * @remarks 无.
 * @attention 无.
 */
int IMP_AO_CacheSwitch(int audioDevId, int aoChn, int cache_en);
```

如果需要用来进行网络放音防抖，建议打开 Cache 功能（默认是打开的），并且适当调整 frmNum 参数即可。

注意：

IMP_AO_CacheSwitch 需要在 IMP_AO_EnableChn 之后调用

3.10 编译内核出错

编译内核出现如下现象

现象 1:

Load Address: ffffffff

现象 2:

Load address error, please upgrade \"mkimage\" command and recompile

如果出现如上现象，mkimage 工具版本比较老，需要重新更新到新版本

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

3.11 帧率不足

查找帧率不足的问题首先需要判断是 ISP 丢帧还是编码丢帧；

(1)摄像头实际的中断：

执行 cat /proc/jz/isp/isp-w02;sleep 1;cat /proc/jz/isp/isp-w02; 计算间隔 1 秒执行 isp-w02 得 到的当前帧号的差值是不是等于设置的帧率。

(2)判断编码帧率不足的方法

执行./impdbg --enc_info;sleep 10;./impdbg --enc_info; 计算间隔 10 秒执行 impdbg 得到的 df 统计的丢帧数的差值，这个差值除以 10 就是每秒丢帧数。

(3) 实际遇到的问题

Q：主码流的帧率只有设置的一半：

```
if [ ${ZRT_ENV_SOC_TYPE} = SOC_T31Z ]; then
    CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdini
    BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
    ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;adc_value=100;eenv;"
    SOC_TYPE=t31
    RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
fi

if [ ${ZRT_ENV_SOC_TYPE} = SOC_T31A ]; then
    CMDLINE="console=${UART},115200n8 mem=80M@0x0 rmem=48M@0x5000000 root=/dev/ram0 rw rdini
    BOOTINFO="kernel=5M@0x98000 rootfs=3M@0x598000"
    ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;eenv;"
    SOC_TYPE=t31
    RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
fi

SENSOR_SETTING_MODE0="../sensor_calibration_bin/${ZRT_ENV_SENSOR}_${SOC_TYPE}.bin"
SENSOR_AE_TABLE="../sensor_start_ae_table/ae_table_${ZRT_ENV_SENSOR}.bin"

cd ${ZRT_ENV_TOOLS_DIR}/make_tag/sensor_start_ae_table/
make clean && make

cd ${ZRT_ENV_TOOLS_DIR}/make_tag/sensor_init_setting/
make ${ZRT_ENV_SENSOR}_${SOC_TYPE}_init
cp ${ZRT_ENV_SENSOR}_${SOC_TYPE}_init.bin ${ZRT_ENV_TOOLS_DIR}/make_tag/tag_generator/

cd ${ZRT_ENV_TOOLS_DIR}/make_tag/tag_generator/
NORMAL 鹏zrt_t31   tools/make_tag/tag_generator/build_tag_t31.sh
```

```
struct chn_conf chn[FS_CHN_NUM] = {
    {
        .index = CH0_INDEX,
        .enable = CHN0_EN,
    .payloadType = IMP_ENC_PROFILE_HEVC_MAIN,
        .fs_chn_attr = {
            .pixFmt = PIX_FMT_NV12,
            .outFrmRateNum = SENSOR_FRAME_RATE_NUM,
            .outFrmRateDen = SENSOR_FRAME_RATE_DEN,
            .nrVBs = 2,
            .type = FS_PHY_CHANNEL,

            .crop.enable = CROP_EN,
            .crop.top = 0,
            .crop.left = 0,
            .crop.width = SENSOR_WIDTH,
            .crop.height = SENSOR_HEIGHT,

            .scaler.enable = 0,

            .picWidth = SENSOR_WIDTH,
            .picHeight = SENSOR_HEIGHT,
          },
        .framesource_chn =  { DEV_ID_FS, CH0_INDEX, 0},
        .imp_encoder = { DEV_ID_ENC, CH0_INDEX, 0},
    },
鹏zrt_t31⚡   userland/4.7.2/imp-t31/samples/libimp-samples/sample-common.c
```


Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

A：找到上两张图所在位置，确定 nrvbs 设置参数是否一致，这里两处的 nrvbs 必须设置一致。如果要改 cmdline 的 nrvbs。可以用如下命令设置，然后重启即可生效：

```
tag_env_info --set HW nrvbs 2
```

Q：cat /proc/jz/isp/isp-w02;sleep 1;cat /proc/jz/isp/isp-w02 差值不等于设置帧率

A：需要查一下 sensor 驱动的 xx_set_fps 函数，确定 pclk，vts，hts 等参数设置的是否正确。

3.12 无法获得内核缓存帧

内核缓存的帧数由 cmdline 中 nrvbs 参数决定，当我们设定 nrvbs 大于 2 的时候，有可能会出现编码后的数据里面，取内核缓存帧只取了一部分，并且应用程序配置的 nrvbs 参数和 cmdline 的参数一致。

主要的原因是由于内核帧是已经缓存好的，framesource 取帧数据的时候取得比较快，但是编码默认的 stream cnt 一般是 2，也就是编码码流缓存 buffer 个数是 2。所以如果编码处理的慢了，或者应用程序没有及时取走，就会出现 framesource 取到的帧数据无法及时处理从而丢弃的现象。

解决此问题，需要通过调用以下 API，nrMaxStream 值需要和 nrvbs 参数值一致。

```
/**
 * @fn int IMP_Encoder_SetMaxStreamCnt(int encChn, int nrMaxStream)
 *
 * 设置码流缓存Buffer个数
 *
 * @param[in] encChn 编码Channel号,取值范围: [0, @ref NR_MAX_ENC_CHN - 1]
 * @param[in] nrMaxStream 码流Buffer数,取值范围: [1, @ref NR_MAX_ENC_CHN_STREAM]
 *
 * @retval 0 成功
 * @retval 非0 失败
 *
 * @remarks 由于码流缓存Buffer个数在通道创建时就已经固定，因此次API需要在通道创建之前调用。
 * @remarks 若通道创建之前不调用此API设置码流缓存Buffer个数，则使用SDK默认的buffer个数。
 *
 * @attention 无
 */
int IMP_Encoder_SetMaxStreamCnt(int encChn, int nrMaxStream);
```

3.13 编译的 boot 无法运行

在使用过程中会遇到编译的卡起 boot，或者 normal boot 无法运行，出现在 boot 启动卡死情况，或者出现进入到 boot 入口就没有打印的情况。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
DDR Controller init
DDRC_STATUS          0x80000001
DDRC_CFG             0x0a288a40
DDRC_CTRL            0x0000011c
DDRC_LMR             0x00400008
DDRC_DLP             0x00000000
DDRC_TIMING1         0x040e0806
DDRC_TIMING2         0x02170707
DDRC_TIMING3         0x2007051e
DDRC_TIMING4         0x1a240031
DDRC_TIMING5         0xff060405
DDRC_TIMING6         0x32170505
DDRC_REFCNT          0x00f25a01
DDRC_MMAP0           0x000020fc
DDRC_MMAP1           0x00002400
DDRC_REMAP1          0x03020d0c
DDRC_REMAP2          0x07060504
DDRC_REMAP3          0x0b0a0908
DDRC_REMAP4          0x0f0e0100
DDRC_REMAP5          0x13121110
DDRC_AUTOSR_EN       0x00000000
sdram init finished
SDRAM init ok
board_init_r
image entry point: 0x80100000
```

一般出现这种情况大多数原因是由于编译的 boot 超出范围导致。Uboot 分区大小 256K，默认的空间分配如下：

SPL 32K	Uboot 216K	ENV 8K

当出现编译后的 uboot 超出 216K 范围的时候，就会出现 uboot 无法运行的情况，此时需要针对 uboot 进行裁剪，或者需要调整分区大小。

以下是对应每个块的配置参数信息，位于 u-boot/include/configs/isvp_t23.h 中：

```
#ifdef CONFIG_SPL_SFC_NAND
#define CONFIG_SPL_PAD_TO       27648 /* equal to spl max size in M200 */
#define CONFIG_SPL_MAX_SIZE     (27 * 1024)
#else
#define CONFIG_SPL_PAD_TO       32768 /* equal to spl max size in M200 */
#define CONFIG_SPL_MAX_SIZE     (32 * 1024)
#endif

#define CONFIG_UBOOT_OFFSET     CONFIG_SPL_MAX_SIZE
#define CONFIG_MMC_RAW_UBOOT_OFFSET (CONFIG_UBOOT_OFFSET / 1024 + 17)
#define CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR (CONFIG_MMC_RAW_UBOOT_OFFSET * 2)//0x5A //wli changed 0x20 /* 16KB offset */
#define CONFIG_SYS_U_BOOT_MAX_SIZE_SECTORS  0x400 /* 512 KB */
```

SPL大小配置

```
#ifdef CONFIG_SFC_NAND
#define CONFIG_SYS_MONITOR_LEN      (512 << 10)
#else
#define CONFIG_SYS_MONITOR_LEN      (216 * 1024)
#endif
#define CONFIG_SYS_MALLOC_LEN       (32 * 1024 * 1024)
#define CONFIG_SYS_BOOTPARAMS_LEN   (128 * 1024)

#define CONFIG_SYS_SDRAM_BASE       0x80000000 /* cached (KSEG0) address */
#define CONFIG_SYS_SDRAM_MAX_TOP    0x84000000 /* don't run into IO space */
#define CONFIG_SYS_INIT_SP_OFFSET   0x400000
#define CONFIG_SYS_LOAD_ADDR        0x82000000
```

Uboot大小配置

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
566 /**
567  * Environment
568  */
569 #ifdef CONFIG_ENV_IS_IN_MMC
570 #define CONFIG_SYS_MMC_ENV_DEV      0
571 #define CONFIG_ENV_SIZE         (32 << 10)
572 #define CONFIG_ENV_OFFSET       (CONFIG_SYS_MONITOR_LEN + CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR * 512)
573 #elif CONFIG_ENV_IS_IN_SPI_FLASH
574 #define CONFIG_ENV_SECT_SIZE    (1024 * 8)
575 #define CONFIG_ENV_SIZE         (1024 * 8)
576 #define CONFIG_ENV_OFFSET       (CONFIG_SYS_MONITOR_LEN + CONFIG_UBOOT_OFFSET)
577 #endif
578
579 /**
580  * SPL configuration
581  */
582 #define CONFIG_SPL
583 #define CONFIG_SPL_FRAMEWORK
```

Env大小配置

在编译之后，可以通过 ls -hl 查看 u-boot.bin 文件的大小，这个就是实际的 uboot 块的大小。只要这个没有超出 216K 即可。

```
-rw-rw-r--   1 wqyan wqyan  28K 11月 12 10:11 MAINTAINERS
-rwxrwxr-x   1 wqyan wqyan  23K 11月 12 10:11 MAKEALL
-rw-rw-r--   1 wqyan wqyan  32K 11月 12 10:11 Makefile
-rwxrwxr-x   1 wqyan wqyan 4.2K 11月 12 10:11 mkconfig
drwxrwxr-x   3 wqyan wqyan 4.0K 11月 12 10:11 nand_spl
drwxrwxr-x   2 wqyan wqyan 4.0K 12月 21 20:45 net
drwxrwxr-x   6 wqyan wqyan 4.0K 12月 21 20:38 post
-rw-rw-r--   1 wqyan wqyan 198K 11月 12 10:11 README
-rw-rw-r--   1 wqyan wqyan 2.5K 11月 12 10:11 rules.mk
-rw-rw-r--   1 wqyan wqyan   17 11月 12 10:11 snapshot.commit
drwxrwxr-x   8 wqyan wqyan 4.0K 12月 21 20:45 spl
-rw-rw-r--   1 wqyan wqyan  33K 12月 21 20:45 System.map
-rw-rw-r--   1 wqyan wqyan  39M 12月 21 17:57 tags
drwxrwxr-x   5 wqyan wqyan 4.0K 12月 21 20:38 test
drwxrwxr-x  17 wqyan wqyan 4.0K 12月 21 20:45 tools
-rwxrwxr-x   1 wqyan wqyan 1.1M 12月 21 20:45 u-boot
-rwxrwxr-x   1 wqyan wqyan 209K 12月 21 20:45 u-boot.bin
-rw-rw-r--   1 wqyan wqyan  867 12月 21 20:38 u-boot.lds
-rw-rw-r--   1 wqyan wqyan 185K 12月 21 20:45 u-boot.map
-rwxrwxr-x   1 wqyan wqyan 649K 12月 21 20:45 u-boot.srec
-rw-rw-r--   1 wqyan wqyan 241K 12月 21 20:45 u-boot-with-spl.bin
-rw-rw-r--   1 wqyan wqyan 258K 12月 21 20:40 u-boot-with-spl-mbr-gpt.bin
```

当编译成 SD 卡起 boot 的时候，通过上面方式查看 boot 并没有超出分区大小，但是还是起不来，一是需要查看编译 boot 的配置是否正确，二是需要查看一下 SD 卡的参数是否正确，CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR 参数是否正确，需要参考如下修改：

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
#ifdef CONFIG_SPL_SFC_NAND
#define CONFIG_SPL_PAD_TO       27648 /* equal to spl max size in M200 */
#define CONFIG_SPL_MAX_SIZE     (27 * 1024)
#else
#define CONFIG_SPL_PAD_TO       32768 /* equal to spl max size in M200 */
#define CONFIG_SPL_MAX_SIZE     (32 * 1024)
#endif

#define CONFIG_UBOOT_OFFSET     CONFIG_SPL_MAX_SIZE
#define CONFIG_MMC_RAW_UBOOT_OFFSET (CONFIG_UBOOT_OFFSET / 1024 + 17)
#define CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR (CONFIG_MMC_RAW_UBOOT_OFFSET * 2) /0x5A //wli changed 0x20 /* 16KB offset */
#define CONFIG_SYS_U_BOOT_MAX_SIZE_SECTORS  0x400 /* 512 KB */

#define CONFIG_SPL_BOARD_INIT
#define CONFIG_SPL_LIBGENERIC_SUPPORT
#define CONFIG_SPL_GPIO_SUPPORT
```

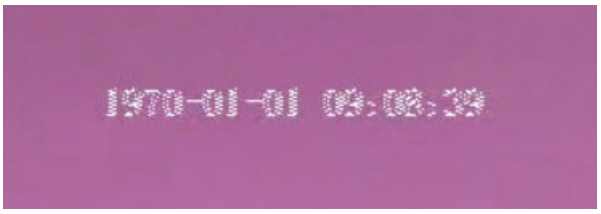
SD 卡启参数配置

3.14 OSD 叠加 BMP 图片显示异常

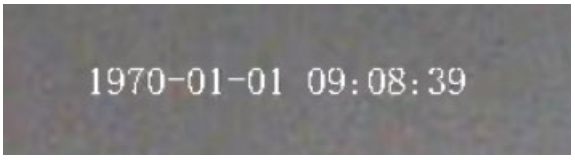
在对 OSD 模块进行叠加 BMP 图片的时候，参考 sample 中的 logo 方式进行叠加，所有的宽高信息，以及格式配置均正确的情况下，出现叠加存在问题。如下：

以下图片是 PC 工具打开的情况，分辨率 230*20 格式为 RGB555LE。在 PC 上显示是正常的。

代码参数配置也正确，但是叠出来的图片出现以下情况：



出现上述情况的主要原因是由于 BMP 的叠加是由硬件 IPU 进行叠加，硬件单元一般对宽有对齐要求，需要按 8 对齐，230*20 的分辨率，按 8 对齐后是 232*20。当重新修改分辨率后，并且原始数据后面补齐两个像素数据后，叠加后的图像就正常了，如下：



注意：

BMP 的图片叠加由于硬件要求，分辨率的宽需要按 8 像素对齐，高没有严格要求。

3.15 卡起 uboot 单独更新某个分区后系统无法启动

在使用卡起 uboot 对设备进行独立分区升级的时候，更新完成后，出现系统无法启

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

动的情况，但是将整个固件全部重烧，系统可以起来，只有单独烧录某个分区的时候起不来。如下，对 rootfs 进行升级后，系统无法启动。

```
isvp_t31# sf probe
the manufacturer ef
SF: Detected W25Q128

--->probe spend 4 ms
isvp_t31# sf erase 0x2E8000 0x280000
SF: 2621440 bytes @ 0x2e8000 Erased: OK
--->erase spend 6938 ms
isvp_t31# mw.b 0x80600000 0xff 0x280000
isvp_t31# fatload mmc 0 rootfs.cpio.lzo
** No boot file defined **
isvp_t31# fatload mmc 0 0x80600000 rootfs.cpio.lzo
reading rootfs.cpio.lzo
1779786 bytes read in 86 ms (19.7 MiB/s)
isvp_t31# sf write 0x80600000 0x2E8000 0x280000
SF: 2621440 bytes @ 0x2e8000 Written: OK
--->write spend 3655 ms
isvp_t31#
isvp_t31#
isvp_t31# reset
reset in 4ms
Ver:20201219-Turret
```

经过调试发现分区信息，以及数据内容并没有错误，但是还是出现无法启动的情况，一般是由于分区大小和 flash 的擦除块大小不对齐导致。

出现上述问题的分区大小如下：

```
if [ ${ZRT_ENV_SOC_TYPE} = SOC_T31Z ]; then
    CMDLINE="console=${UART},115200n8 mem=32M@0x0 rmem=32M@0x2000000 root=/dev/ram0 rw rdinit=/linuxrc
    mtdparts=jz_sfc:256K(boot),352K(tag),2368K(kernel),2560K(rootfs),2048K(recovery),5120K(system),384K(config),16M@0(all)
    lpj=6955008 ${QUIET}"
    BOOTINFO="kernel=2368K@0x98000 rootfs=2560K@0x2E8000"
    ENVINFO="senv;[HW];init_vw=1920;init_vh=1080;nrvbs=3;mode=0;eenv;"
    SOC_TYPE=t31
    RISCV_FW="../riscv_fw/riscv_fw_${ZRT_ENV_SENSOR}.bin"
fi
```

Rootfs 的偏移地址为 2976K，可被 32K 整除，但是无法被 64K 整除，一般 Flash 默认的簇大小一般是 64K 大小。所以当出现单独分区烧录的时候，在擦除 rootfs 的时候，会把 kernel 分区的末尾 32K 一并擦除了，导致无法启动。

对应的 flash 的参数如下：

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
312     {
313         .name           = "W25Q128",
314         .id_manufactory = 0xef4018,
315         .page_size       = 256,
316         .sector_size      = (64 * 1024),
317         .addr_size = 3,
318         .size           = (16 * 1024 * 1024),
319         .quad_mode = {
320             .dummy_byte = 6,
321             .RDSR_CMD = CMD_RDSR_1,
322             .WRSR_CMD = CMD_WRSR,
323             .RDSR_DATE = 0x2,//the data is write the spi status register for QE bit
324             .RD_DATE_SIZE = 1,
325             .WRSR_DATE = 0x0200,//his bit should be the flash QUAD mode enable
326             .WD_DATE_SIZE = 1,
327             .cmd_read = CMD_QUAD_IO_FAST_READ,
328 #ifdef CONFIG_JZ_SFC
329             .sfc_mode = TRAN_SPI_IO_QUAD,
330 #endif
331         },
332
333     },
334     {
335         .name           = "W25Q256JV",
336         .id_manufactory = 0xef4019,
337         .page_size       = 256,
~/work/zeratul/os/uboot/u-boot/drivers/spi/jz_spi.h[POS=316,40][39%]06/01/21 - 16:36
```

解决方法：

修改卡起 uboot 源码，将 drivers/spi/jz_spi.h 文件中对应的 flash 型号下的 sector_size 修改为可被独立分区偏移整除的大小，如 32K。

再重新编译 uboot 后重新更新对应分区即可。

Copyright® 2005-2023 Ingenic Semiconductor Co., Ltd. All rights reserved.